

Adam Pinkos

CS 4040 – Design and Analysis of Algorithms

Dr. Majid Mirzanezhad

October 28th, 2025

Milestone 2

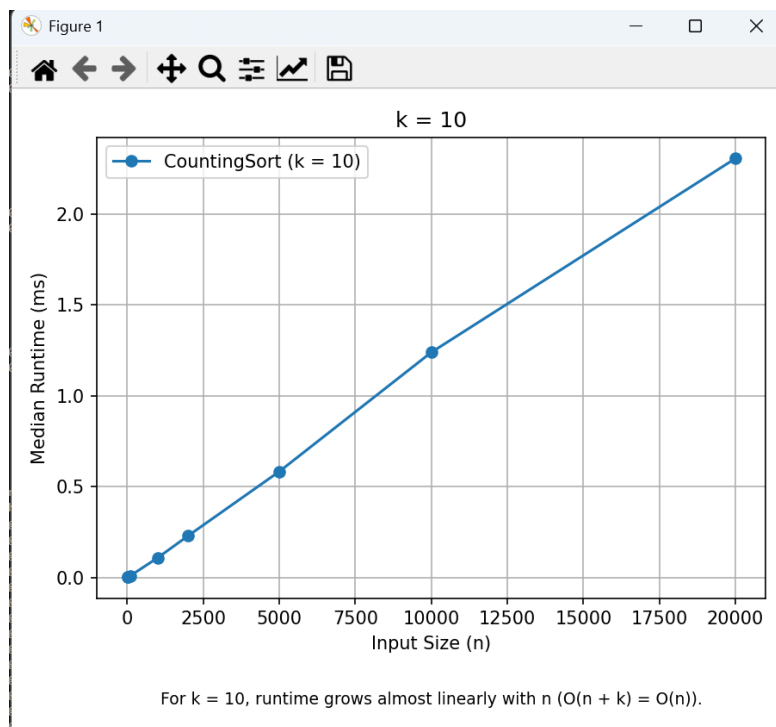
This project program in python explores the efficiency of Counting Sort by measuring how its runtime changes for different input sizes, which is n and key ranges, which is k . Counting Sort is a non-comparison sorting algorithm with a theoretical time complexity of $O(n + k)$, meaning its performance depends on both the number of elements being sorted and the range of possible values. In this experiment, three separate cases were tested. $k = 10$, $k = n$, and $k = n^2$. To observe how increasing k affects execution time. For each configuration, five trials were performed, and the median runtime was recorded. The resulting graphs show how Counting Sort remains efficient for small k , but becomes significantly slower as k grows, confirming the theoretical relationship between n , k , and runtime.

The table below displays the median runtimes in milliseconds across multiple input sizes (n) and maximum key values (k). Each value represents the median of five trials.

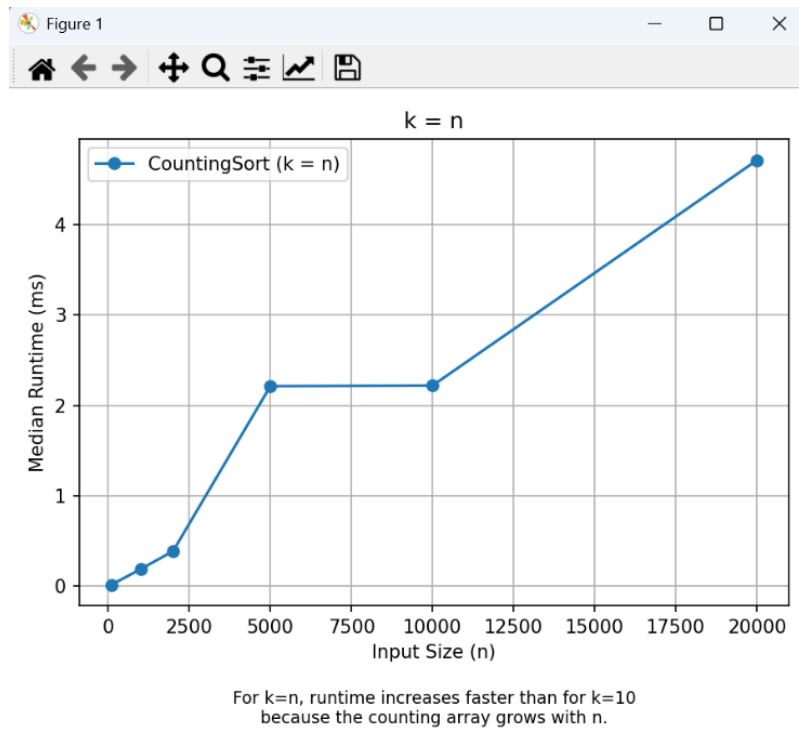
=== Runtime Table (Median of 5 runs) ===			
n	k (max)	Algorithm (params)	Median (ms)
10	10	CountingSort(k=10)	0.003
10	10	CountingSort(k=10)	0.002
10	100	CountingSort(k=100)	0.006
100	10	CountingSort(k=10)	0.010
100	100	CountingSort(k=100)	0.014
100	10000	CountingSort(k=10000)	0.642
1000	10	CountingSort(k=10)	0.112
1000	1000	CountingSort(k=1000)	0.197
1000	1000000	CountingSort(k=1000000)	76.353
2000	10	CountingSort(k=10)	0.224
2000	2000	CountingSort(k=2000)	0.384
2000	1000000	CountingSort(k=1000000)	78.918
5000	10	CountingSort(k=10)	0.598
5000	5000	CountingSort(k=5000)	1.076
5000	1000000	CountingSort(k=1000000)	81.174
10000	10	CountingSort(k=10)	1.197
10000	10000	CountingSort(k=10000)	2.119
10000	1000000	CountingSort(k=1000000)	86.416
20000	10	CountingSort(k=10)	2.402
20000	20000	CountingSort(k=20000)	4.552
20000	1000000	CountingSort(k=1000000)	94.825

For small key sizes such as $k = 10$, the runtimes remain extremely low even as n increases, showing that Counting Sort scales nearly linearly with input size when k is small. However, when k increases to match n , the runtimes grow slightly faster. When $k = n^2$, the runtimes increase dramatically, especially once k exceeds one million, confirming that Counting Sort becomes inefficient when the range of key values dominates the input size. For example, at $n = 1000$ and $k = 1,000,000$, the runtime jumps to over 76 ms. This shows that Counting Sort is highly efficient for small or moderately sized key ranges.

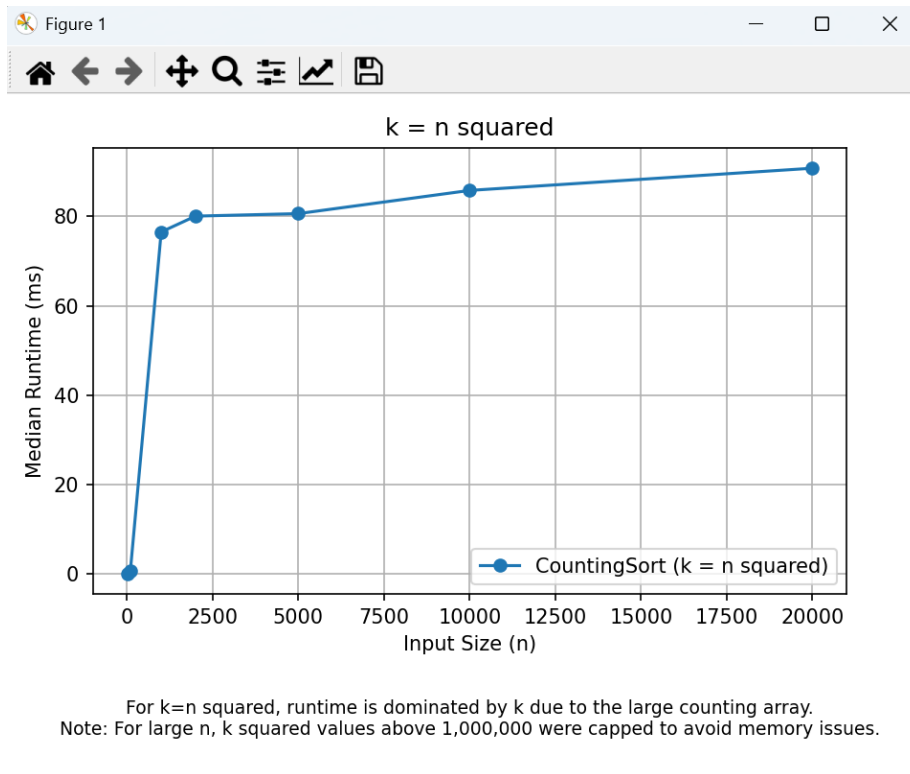
Graphs:



The graph above shows the relationship between input size and median runtime when $k = 10$. The line increases almost perfectly linearly, confirming the expected **$O(n + k)$ performance**, which we found out during lecture that it simplifies to **$O(n)$** when k is a small constant. As the number of elements grows from 10 to 20,000, the runtime rises steadily from nearly 0 ms to just over 2 ms, showing that Counting Sort handles increasing input sizes very efficiently when the key range remains fixed.



The graph above shows the runtime behavior of Counting Sort when the maximum key value k grows proportionally with the input size. Compared to the previous case where $k = 10$, the runtime increases at a noticeably faster rate, reflecting the added cost of initializing and processing a larger counting array for every increment in n . As the input size increases from 10 to 20,000, the runtime rises more sharply, from near zero to roughly 4.5 milliseconds, demonstrating that both n and k contribute significantly to the total runtime. This behavior is consistent with the theoretical complexity $O(n + k)$, since here k grows linearly with n , effectively making the runtime proportional to $O(2n)$.



This graph shows the runtime performance of Counting Sort when the key range k grows quadratically with the input size n . In this case, the runtime increases dramatically at small input sizes and quickly plateaus near 80–90 milliseconds once k reaches the capped value of 1,000,000. This behavior occurs because the $O(n + k)$ complexity becomes dominated by k , meaning that most of the time is spent initializing and processing the large counting array rather than sorting the elements themselves. After reaching the cap, the growth slows since k no longer increases beyond the imposed limit.

The experimental results confirm the theoretical runtime of **Counting Sort**, which operates in $O(n + k)$ time. When k is small or constant, the algorithm performs extremely efficiently, scaling almost linearly with input size. As k increases to match n , runtime growth becomes steeper due to the expanding counting array, though performance remains manageable. However, when $k = n^2$, the runtime becomes dominated by the size of k , causing a sharp rise in execution time and significant memory usage.

Sources Cited

GeeksforGeeks. (2023, September 14). Counting Sort. Retrieved from
<https://www.geeksforgeeks.org/dsa/counting-sort/>

Mirzanezhad, M. (2025). All Lectures. CS 4040/5040 – Design and Analysis of Algorithms, Ohio
University. [Class slides]

W3Schools. (n.d.). *DSA Counting Sort*. Retrieved from
https://www.w3schools.com/dsa/dsa_algo_countingsort.php